0875. Koko Eating Bananas

um | Category: Binary Search | LeetCode: https://leetcode.com/problems/koko-e

Problem

- Piles[i] bananas; each hour Koko eats up to k bananas from one pile. Find the minimum k to finish within h hours.
- Example: piles [3,6,7,11], h = 8 → 4.

Intuition

- If speed k works, any faster speed also works: feasibility is monotonic in k.
- So binary search on the answer space [1, max(pile)] for the first feasible k.

Brute Force

- Try k = 1, 2, ... until feasible: O(max n).

Optimal Approach

- Lo = 1, hi = max(piles).
- Hours(k) = sum of ceil(p / k) = sum (p + k - 1) / k.
- If hours(mid) <= h, hi = mid (mid might be the answer); else lo = mid + 1. Return lo.

Worked Example

```
piles [3,6,7,11], h = 8.

lo=1 hi=11 mid=6  hours=1+1+2+2=6  <=8 -> hi=6
lo=1 hi=6  mid=3  hours=1+2+3+4=10 >8  -> lo=4
lo=4 hi=6  mid=5  hours=1+2+2+3=8  <=8 -> hi=5
lo=4 hi=5  mid=4  hours=1+2+2+3=8  <=8 -> hi=4
answer 4
```

Java Solution

```java
public class KokoEatingBananas {
    public int minEatingSpeed(int[] piles, int h) {
        int lo = 1, hi = 0;
        for (int p : piles) hi = Math.max(hi, p);
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (hours(piles, mid) <= h) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }

    private long hours(int[] piles, int k) {
        long total = 0;
        for (int p : piles) total += (p + (long) k - 1) / k;
        return total;
    }
}
```

Complexity

- Time: O(n log max). Space: O(1).

Pitfalls

- Hours can exceed int range for large piles with small k; sum into a long.